Republic Of Cameroon

Peace-Work-Fatherland

PK Fokam Institute of Excellence

République Du Cameroun

Paix-Travail-Patrie

PK Fokam Institute of Excellence

Project: Wedding Invitation and Gift Management System (WIGMS)

Third Progressive Report

V1.3

GROUP 1		
NAME	MATRICULE	PERCENTAGE PARTICIPATION
TCHOUNGA NGATCHOU MARIE CLAIRE	26S022	17.5%
MASSADO DJOKO AICHA CLAUDIA	26S033	17.5%
NKOMPNANG GUILLAUME JUNIOR	26S017	17.5%
AVOUTIA DONGMO BRAYANN	26S028	17.5%
NZANUM RYAN DZEMBUIN	26S007	15%
Chi Rawlings Ngang	25S021	15%

SUPERVISOR: MR MBIETIEU AMOS

Academic Year: 2025/2026

Table of Contents

1. Introduction 3
2. Gantt Chart 3
3.System Design 4
3.2 Sequence Diagram 4
Deployment Diagram 6
5. Class Diagram 7
6. Work Distribution 7
7. User Interface 8
1.3 Implemented Functions 10
1.3.1 create_person () 10
1.3.3 display_person () 11
1.4 Person's Graphic User Interface 12
1.3 Person Management(Output) 14
1.4 Setting Up a GitHub Repository 15
2.1 Objectives 17
2.2 Category Structure 17
2.3 Implemented Functions 18
2.4 Category's Graphical User Interface 19
2.5 Linked List Explanation 22
2.6 Category Management(Output) 22
3.1 Objectives of The Module 23
3.2 Description of the Module 23
3.3 Implemented Functions 23
3.4 Priority's Graphic User Interface 25
8. Repository 27
9. Tools and Technologies Used 27
10. Difficulties Encountered and Proposed Solutions 28
11. Next Steps of the Project 28
12.Conclusion 28

1. Introduction

The aim of this project is to design and implement a Wedding Invitation and Gift Management System using the C programming language. The system is developed using a modular approach, allowing each component to be built, tested, and improved independently.

This fourth report presents:

- The corrected version of Module 1: Person Management based on supervisor remarks
- The corrected version of Module 2: Category Management based on supervisor remarks
- The implementation of Module 3:Priority Management

Special attention was given to improving the user interface, ensuring better usability before proceeding with advanced modules.

2. Gantt Chart

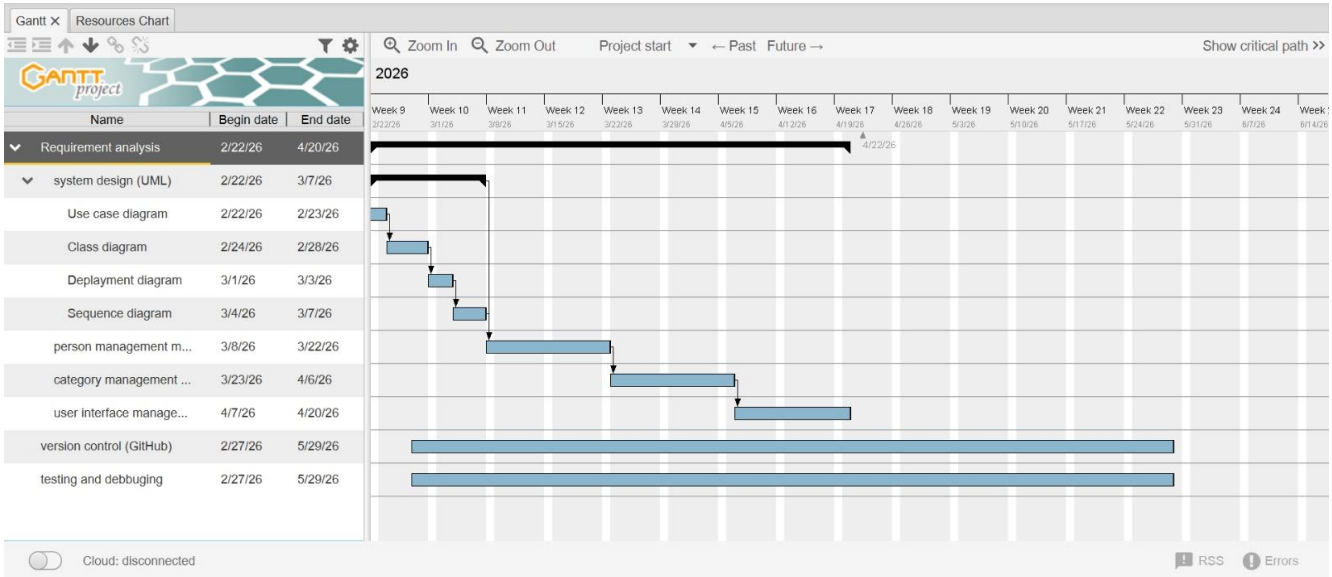


Figure 1: This image represents the project schedule showing the different tasks to be completed over time.

3.System Design

3.1Use Case Diagram

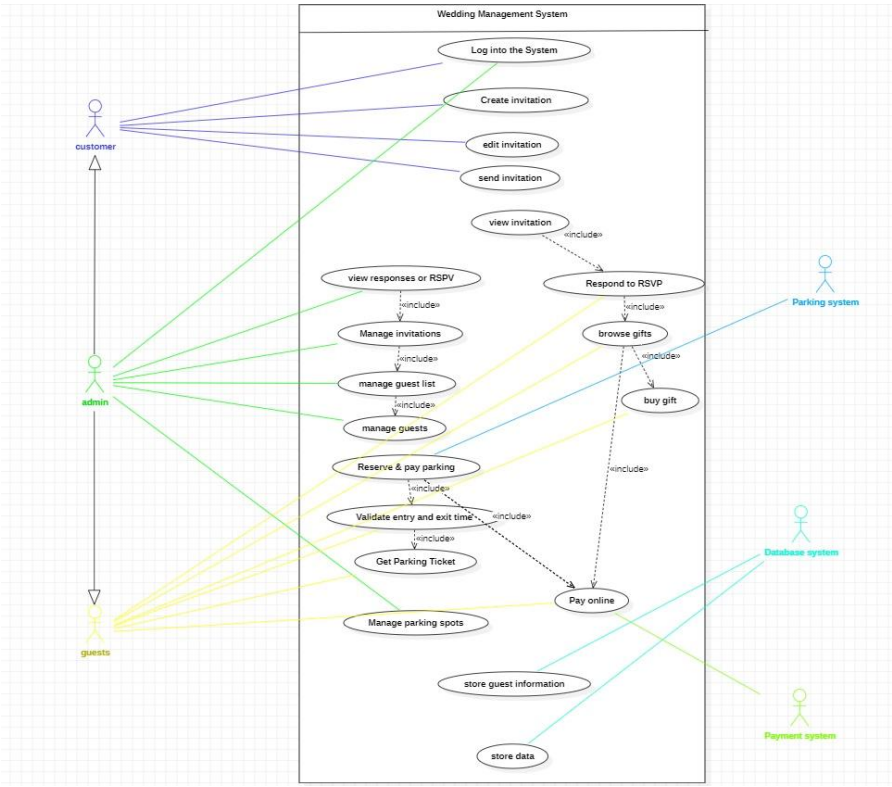


Figure 2: This diagram illustrates the interaction between users and the system. It shows the main functionalities such as managing guests, categories, and invitations, and how different users (admin or user) interact with these features.

3.2 Sequence Diagram

3.2.1 Guest list management sequence

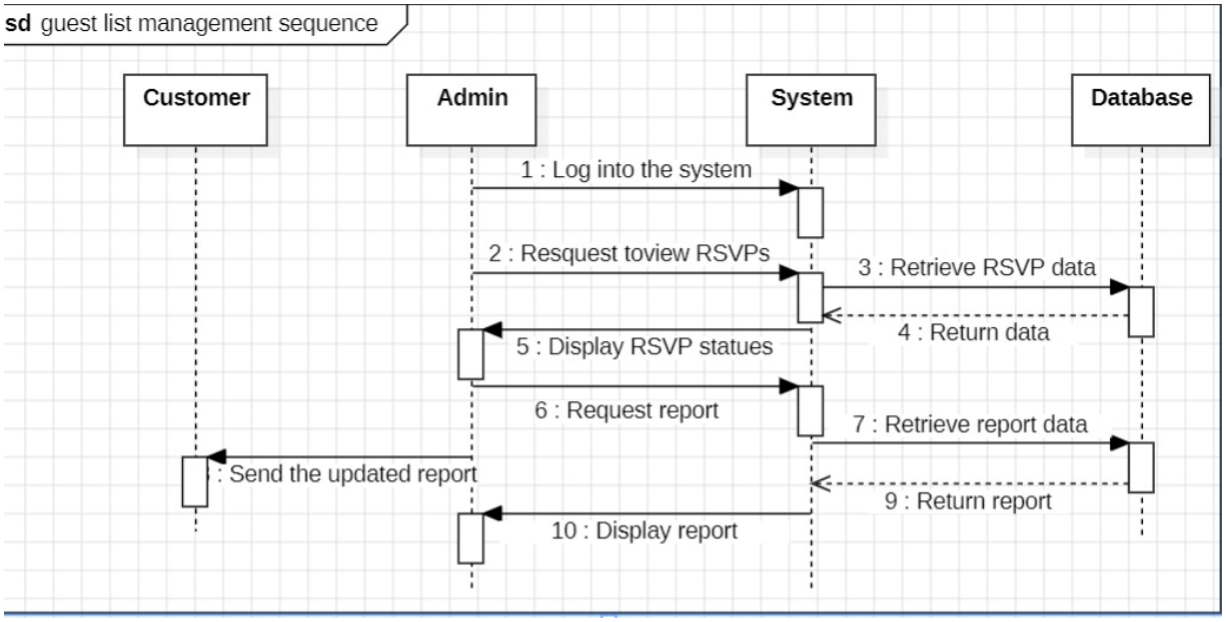


Figure 3: This diagram shows the step-by-step interaction between the user and the system when managing the guest list. It describes how actions like adding, updating, or deleting a guest are processed in sequence

3.2.2 Parking Gestion management sequence

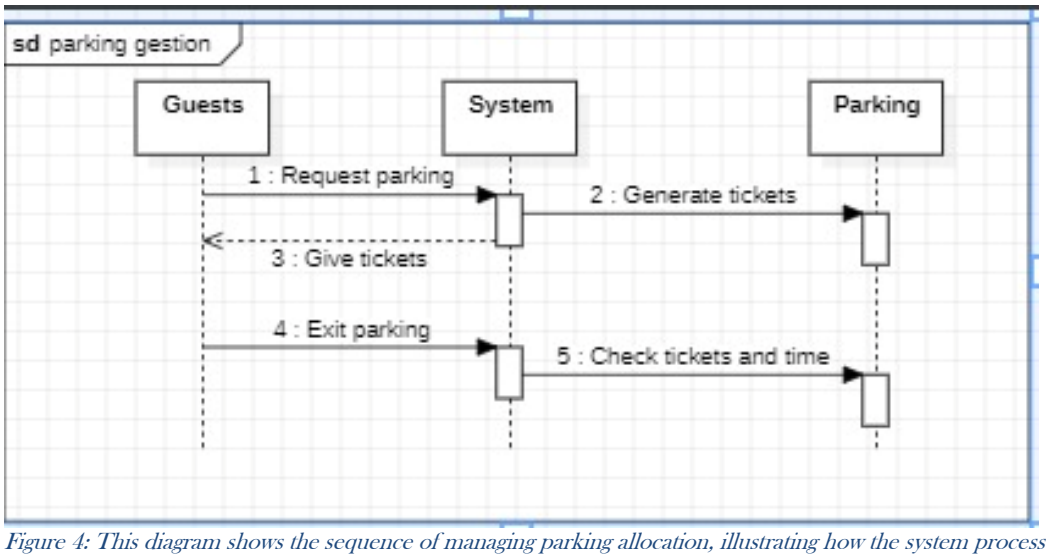


Figure 4: This diagram shows the sequence of managing parking allocation, illustrating how the system processes parking requests for guests.

3.2.3 Invitation creation sequence

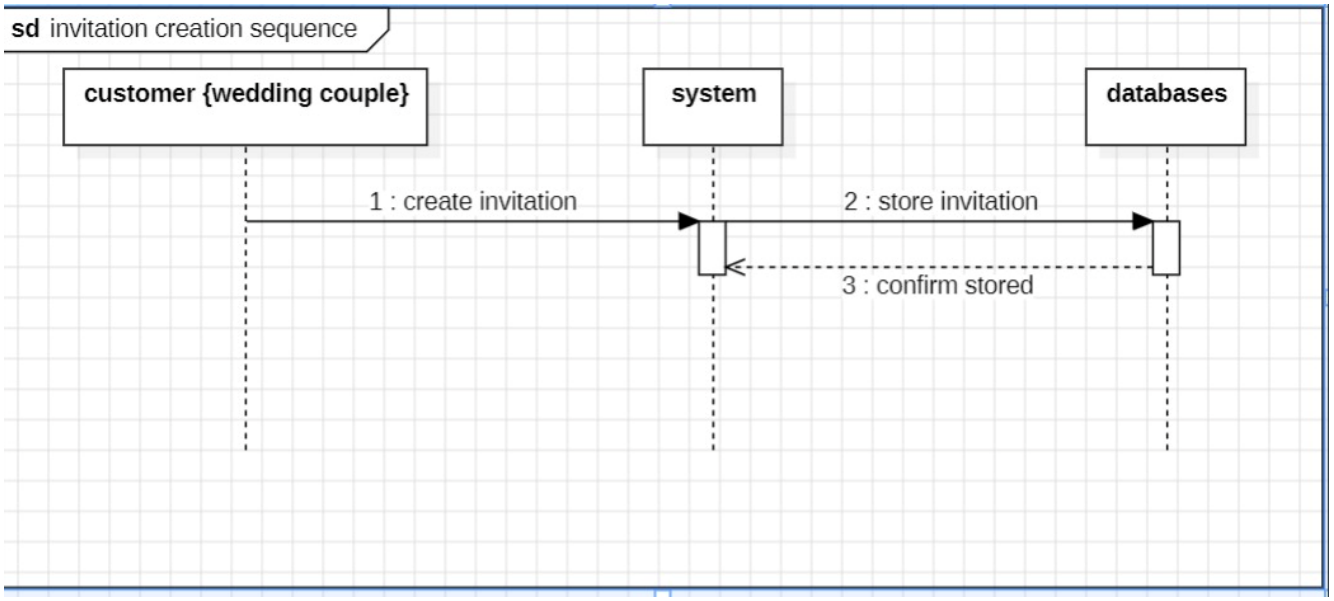


Figure 5: This sequence diagram illustrates the process of creating an invitation, from user input to system processing and final generation.

3.2.4 Respond sequence

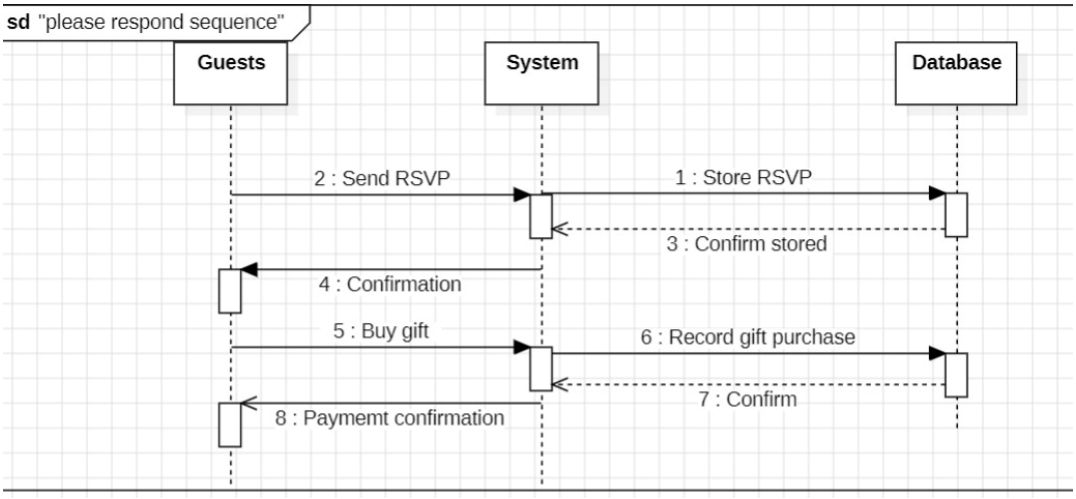


Figure 6: This diagram explains how the system handles guest responses. It shows how a guest accepts or declines an invitation and how the system updates the information accordingly.

Deployment Diagram

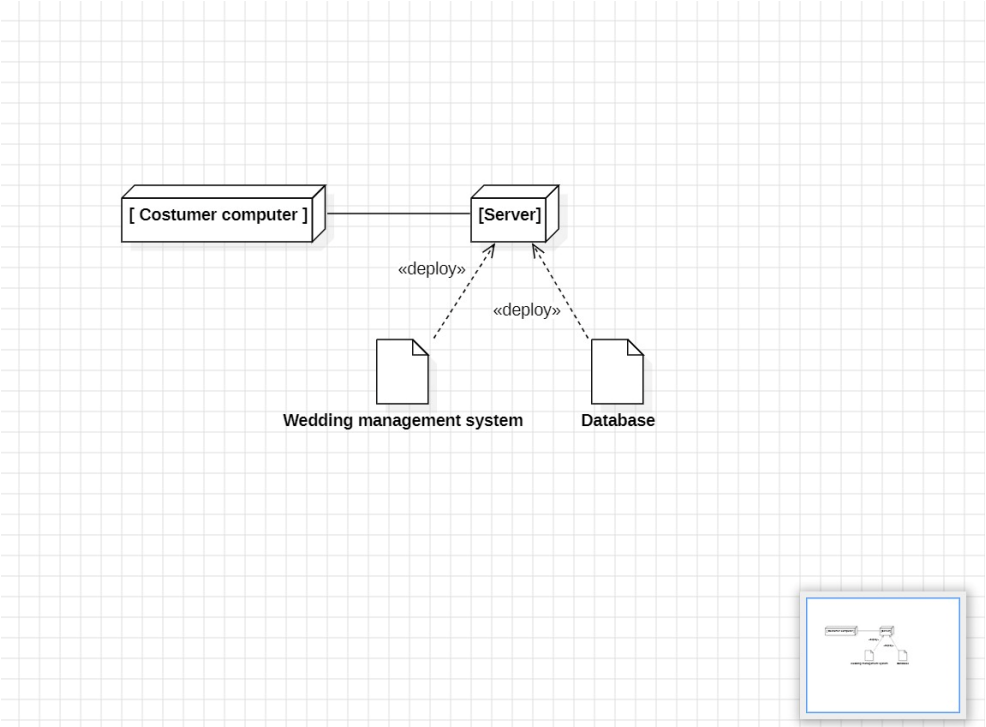


Figure 7: This diagram represents the physical architecture of the system. It shows how the software components are deployed on hardware such as computers and how they communicate with each other.

5. Class Diagram

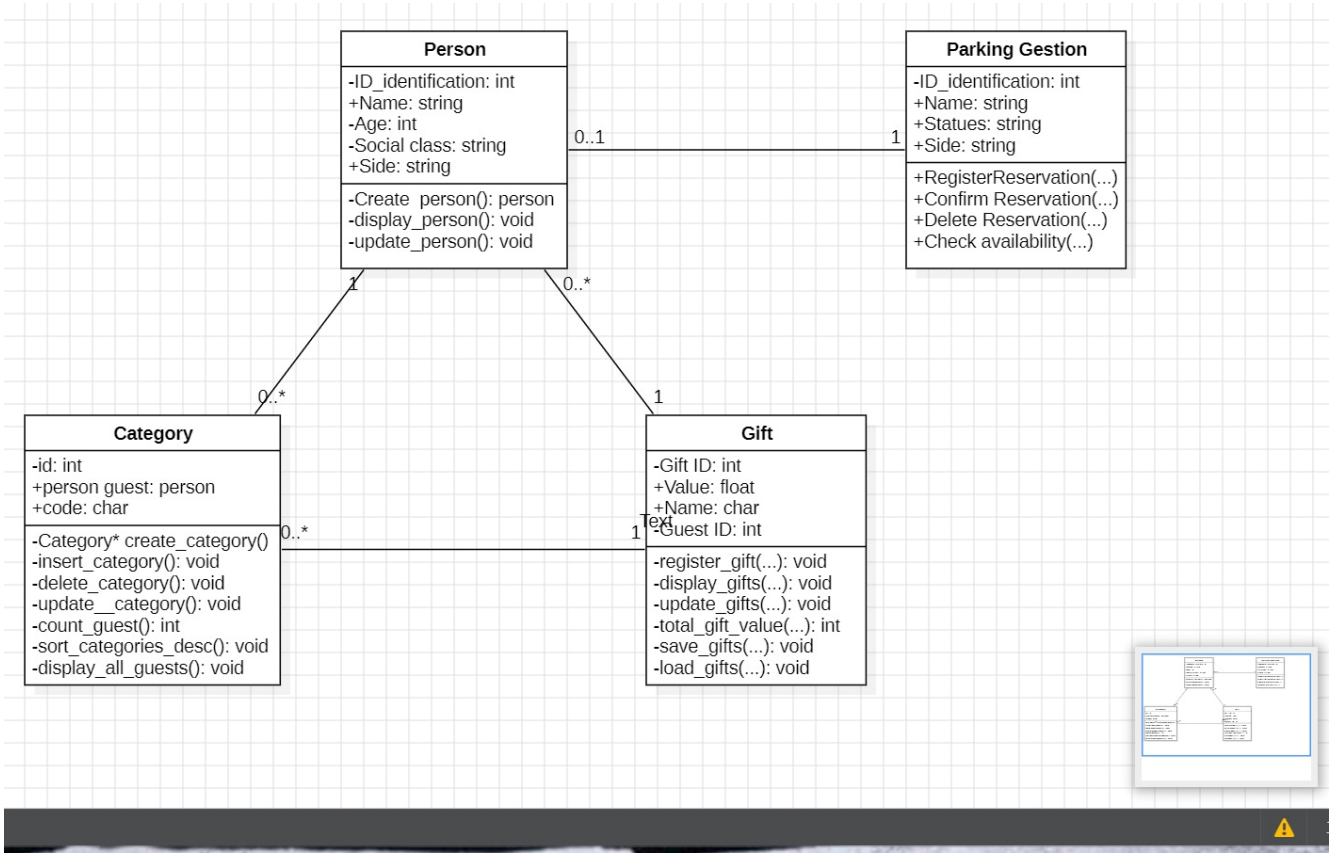


Figure 8: This diagram illustrates the structure of the system by showing different classes (such as Person and Category), their attributes, and the relationships between them.

6. Work Distribution

Group Member	Role	Assigned Tasks
Tchounga Ngatchou Marie Claire	Project Manager and Coordinator	Overall project coordination and writing ,formatting report.
Avoutia Dongmo Brayann	Modules developer	Implementation of some libraries and debugging modules
Nkompnang Guillaume Junior	Modules designer	Graphical user interface design and implementation of remaining librairies
Massado Djocko Aicha	System and Gantt chart designer	Case and sequence diagram design.
Nzanum Ryan Dzembuin	Modificator	Modification of the use case diagram
Chi Rawlings Ngang	Documentation	sequence diagram design

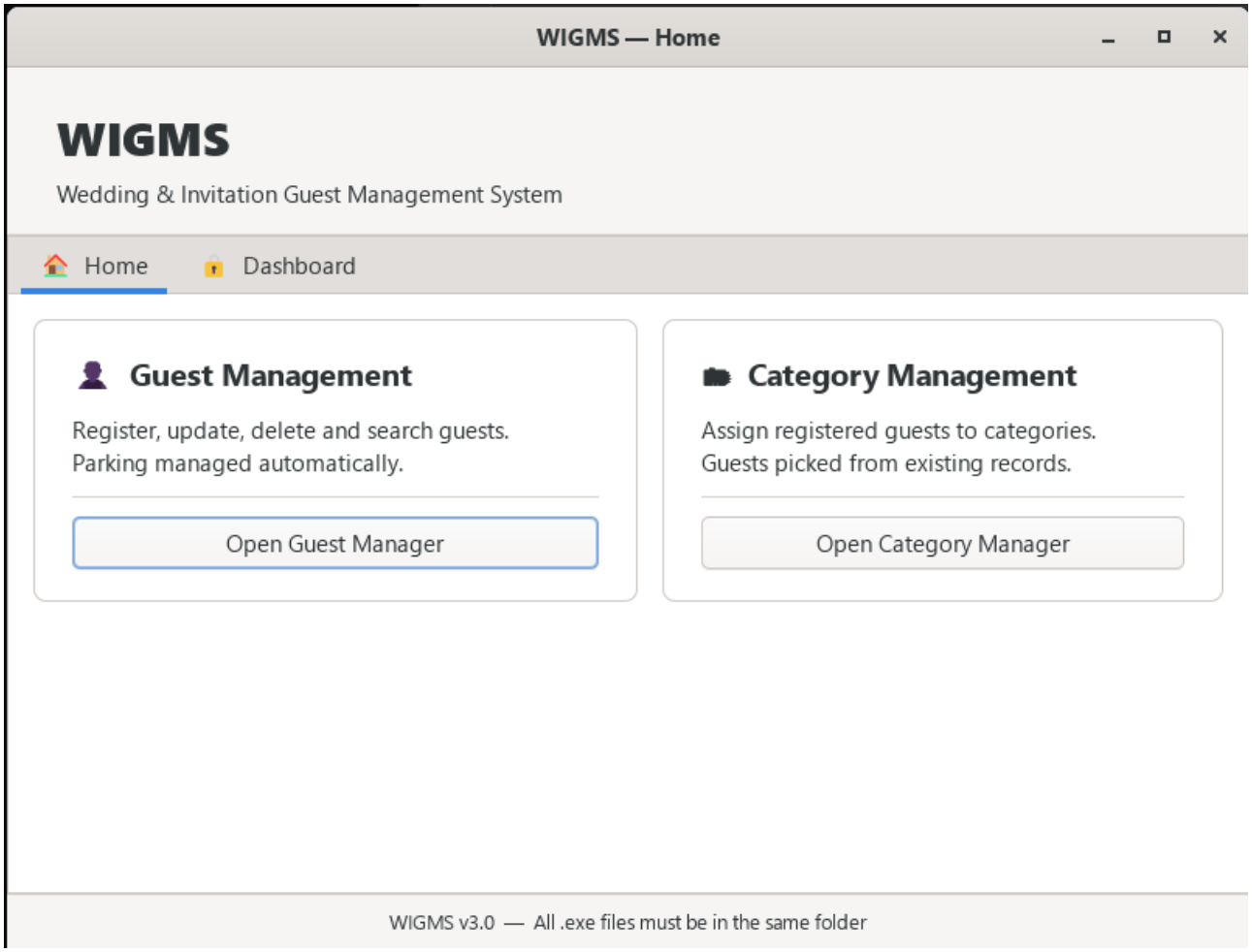
7. User Interface

To improve interaction with the system, a simple user interface was introduced using standard input/output functions in C. The interface allows users to navigate through the program using a menu-driven approach.

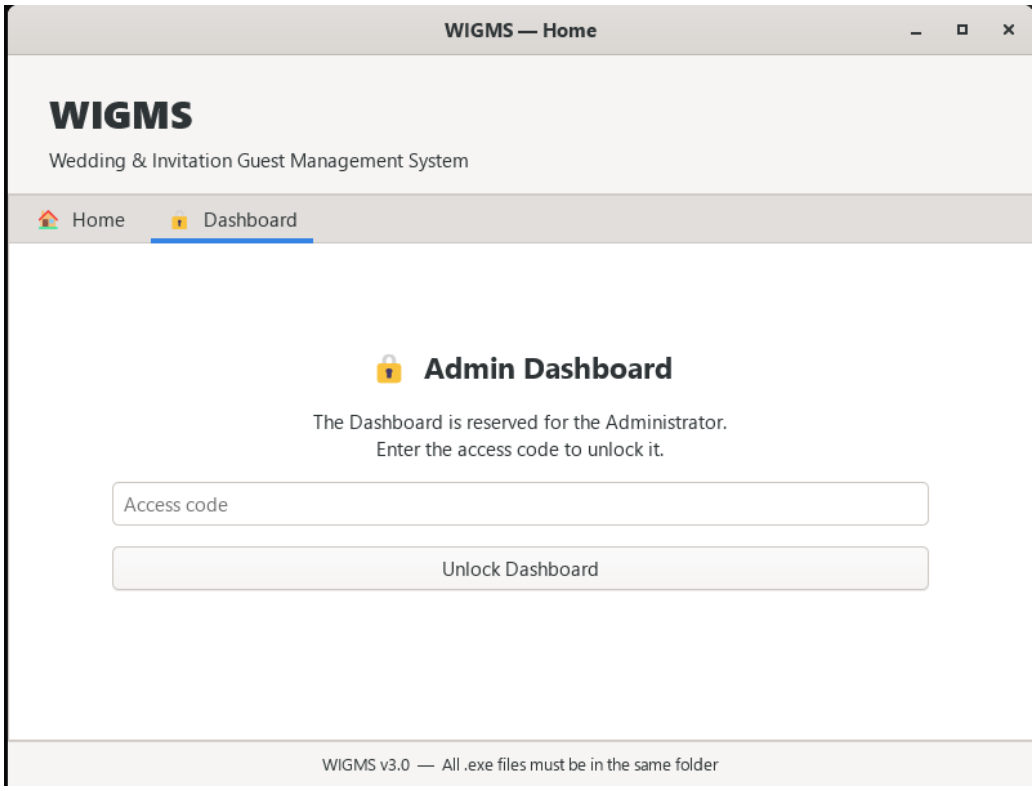
Home page

Here in this program, we can see the home page is handling two main part:

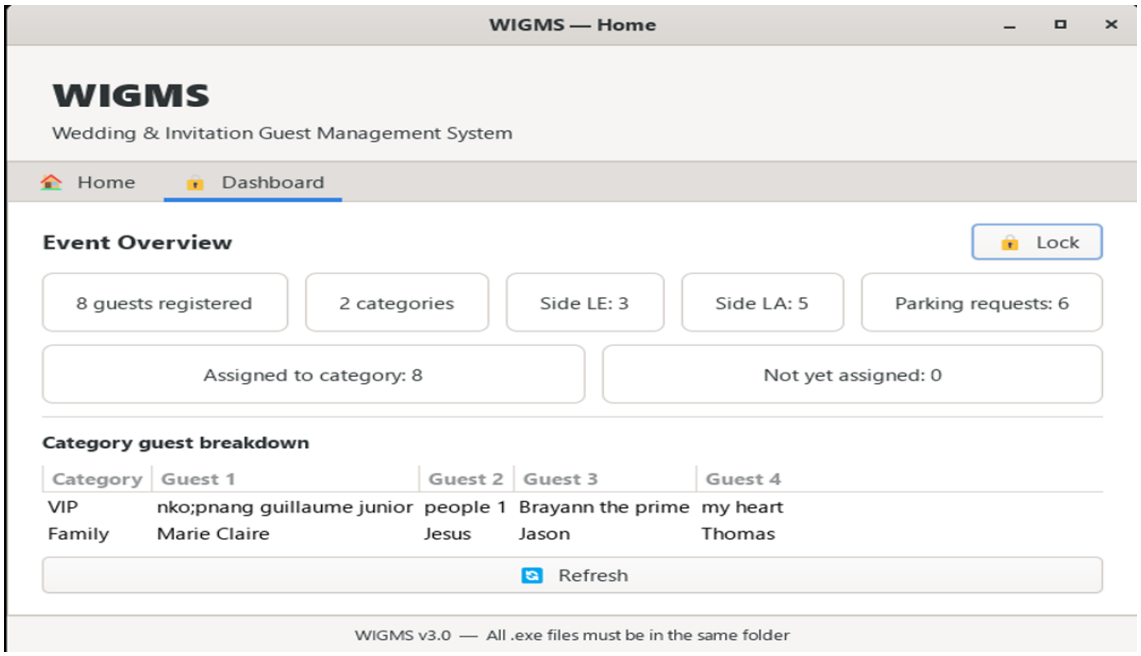
- **The Home:** that's where each user come to connect itself in order to have access to the differents functionalities of the application. Here, since we have just handled two module (and occasionally the third one); we could have access to the person/Guest management system and to the category management system.



- **The Dashboard:** Here, we are handling a general presentation of all the data which are manage by the application through a sort of recapitulative page in order to let the admin of the application have a general view of the data mange by the app. Since the dashboard is a page where the data of all the people are presented, we think that this part should be confidential and be exclusively accessible to the Admin of the app. That's why to access to this page, the user should know the access key of the dashboard, which is "**pokemon 1**". After entering the access key, you could have the access to all the data of the programm as you can see below



This page provides a summary of all system data. It is restricted to admin access and shows an overview of guests and categories.



MODULE 1: PERSON MANAGEMENT

1. 1 Objective of Module

- Define a Person structure to store guest information.
- Implement a function to create a new person.
- Implement a function to display a person's information.
- Implement a function to update a person's information.
- Apply structured programming concepts using C language.

1.2 Person Structure

```c
typedef struct{
    int id;
    char name[100];
    int age;
    char social_class[50];
    side side;
}Person;
```

Figure 9: This image shows the definition of the Person structure in C. It is used to store information about each guest such as ID, name, age, and social class.

1.3 Implemented Functions

1.3.1 create_person ()

```c
Person create_person(){
    Person P;
    printf("\n- - - Guest Input - - -\n");
    printf("ENter an ID\n");
    scanf("%d",&P.id);
    getchar();

    printf("\n- - - Guest Name - - -\n");
    printf("Enter a name\n");
    fgets(P.name,sizeof(P.name),stdin);


    printf("\n- - - Guest AGE - - -\n");
    printf("ENter your Age\n");
    scanf("%d",&P.age);
    getchar();

    printf("\n- - - Guest Social Class - - -\n");
    printf("Enter your Social Class\n");
    fgets(P.social_class,sizeof(P.social_class),stdin);

    printf("\n - - -Guest side- - -\n");
    printf("Enter 0 for LE\n");
    printf("Enter 1 for LA\n");
    scanf("%d",&P.side);
    return P;
```

Figure 10:This code represents the function used to create a new person. It collects user input and initializes a new guest with the provided information.

1.3.2 Create_Person() (Parking)

```c
    printf("\n - - - PARKING - - - \n");

    if (strcmp(P.social_class, "VIP") == 0) {

        do {
            printf("VIP Access: Enter your preferred parking_id (1-100): ");
            scanf("%d", &parking_id);

        } while (1);
        A[0] = parking_id;
        printf("ID %d assigned to VIP.\n", parking_id);
    } else {

        int random_id;
        int slot_found = -1;
        for (int i = 0; i < 100; i++) {
            if (A[i] == 0) {
                slot_found = i;
                break;
            }
    }
        do {
            is_taken = 0;
            random_id = (rand() % 900) + 100;

            for (int i = 0; i < 100; i++) {
                if (A[i] == random_id) {
                is_taken = 1;
                break;
             }
          }
      } while (is_taken);

      if (slot_found != -1) {
          A[slot_found] = random_id;
          printf("Standard Access: You have been assigned ID: %d\n", random_id);
      }
    }

    return P;
}
```

Figure 11: This function extends the person creation process by adding parking information. It allows the system to record whether a guest requires a parking space.

1.3.3 display_person ()

```c
void display_Person(Person P)
{
    int side_choice;
    printf("\n- - - Guest details- - -\n");
    printf("ID:%d\t | Age:%d\t |",P.id,P.age);
    printf("Parking_ID:%d",P.parking_id);

    if(P.side==0)
    {
        printf("side 'LE'\t");
    }
    else
    {
        printf("side 'LA'\t");
    }
    printf("| Name:%s | Social_class:%s ",P.name,P.social_class);
}
```

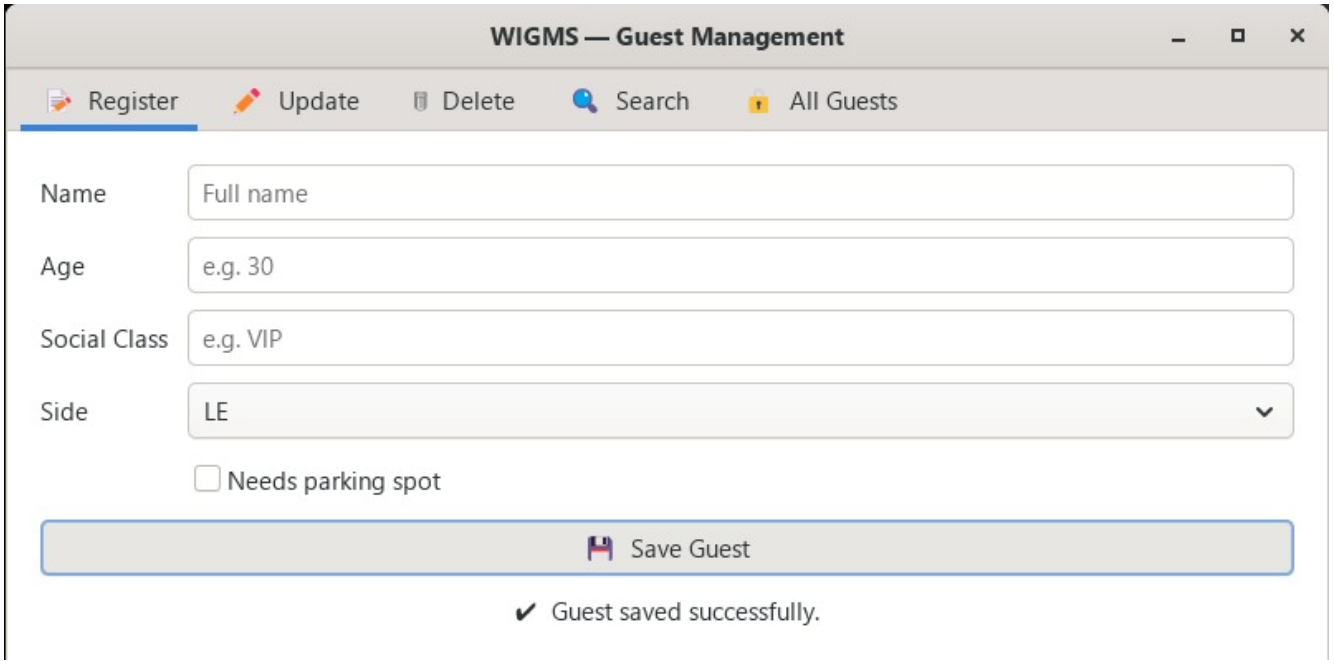
Figure 12: This image shows the function responsible for displaying the details of a person. It outputs the stored information such as name,

age, and social class to the user.

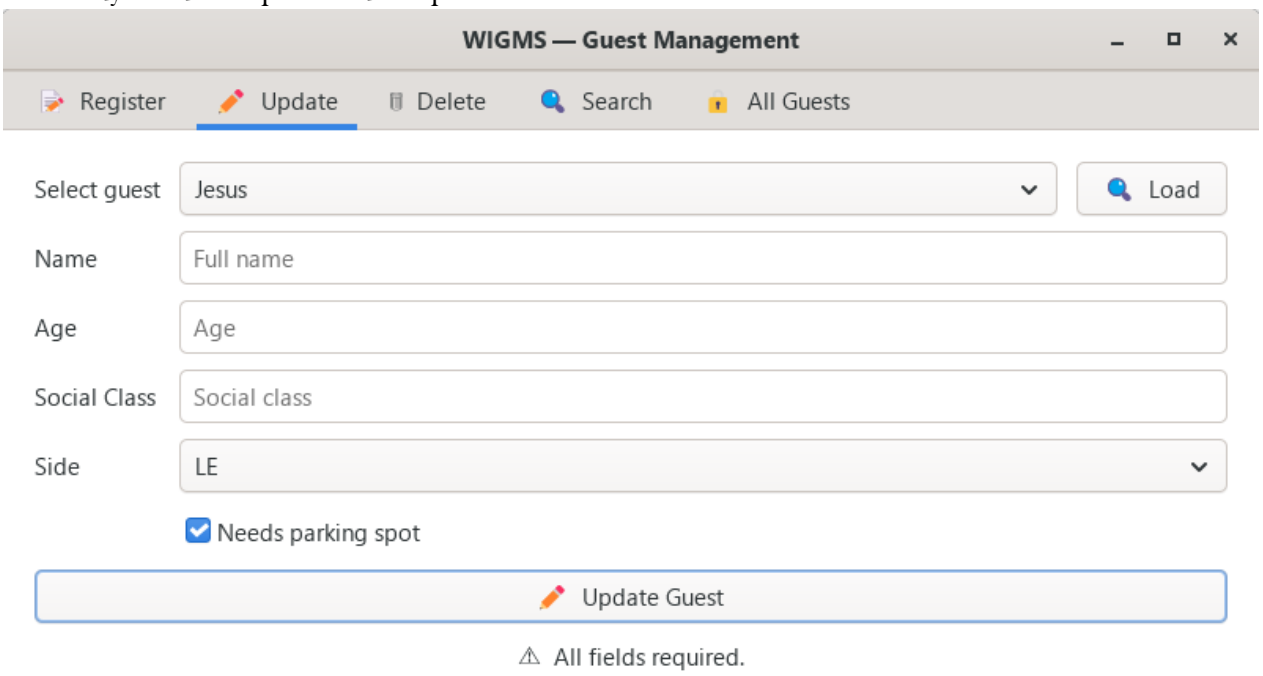
1.4 Person's Graphic User Interface

For the person module graphic user interface, we are handling the regidtration of the different guests that will participate to the event, the modification of their entered paramaters and the suppression of those whose turned down their invitation. In this program, we can do a lot of tasks like;

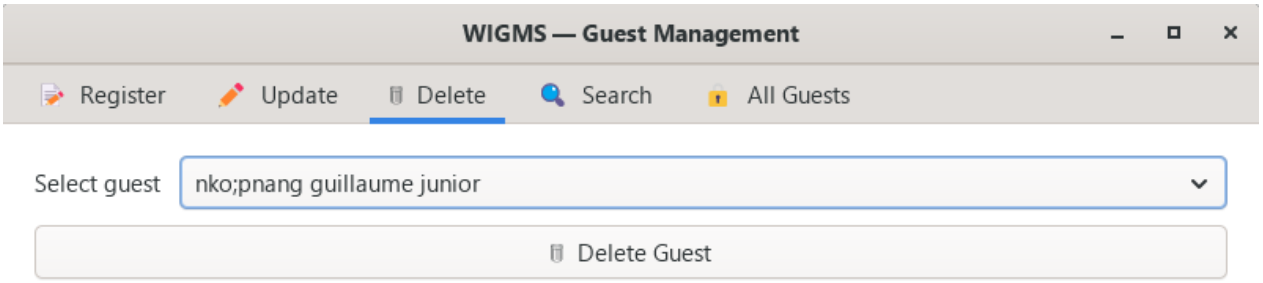
- **The Registration:** here, we are registering the different guests which would like to attend to the ceremony. The user will have to enter his/her entire name, age and social class. We also made ourselves sure to let the ID of each person be automatically generated and to leave a check box to say if he need a parking spot or not. It is as we can see here below:



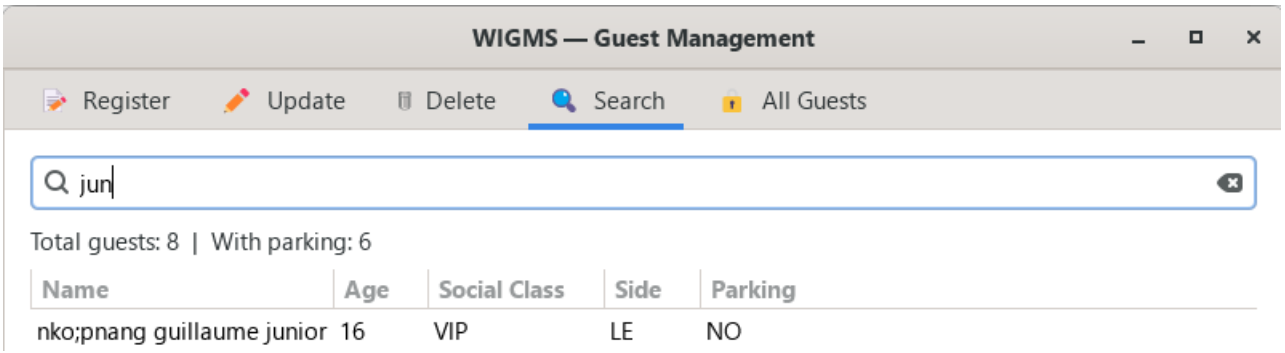
- **Update:** here we have the ability to modify the information on each guest just by entering the name of the of the person in the research space and just modifying the information which are already inside the person description. Like here:



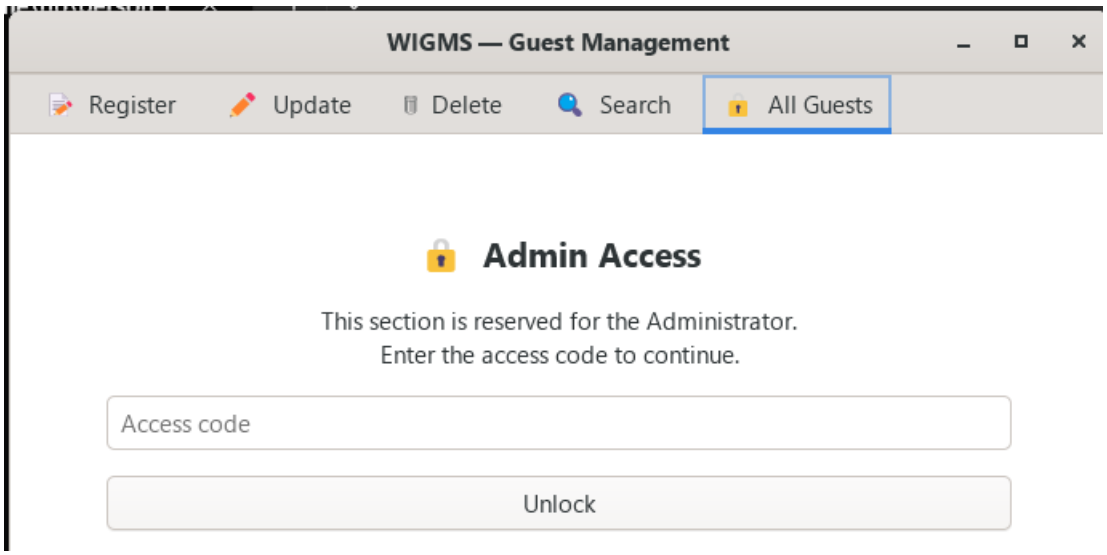
- **Delete: :** It's an option where it's possible for an user to delete an existant guest and the informations about him in the case that you have created a person which where not supposed to exist. It's like on this picture, where you just have the charge of write the name of the person and select delete.



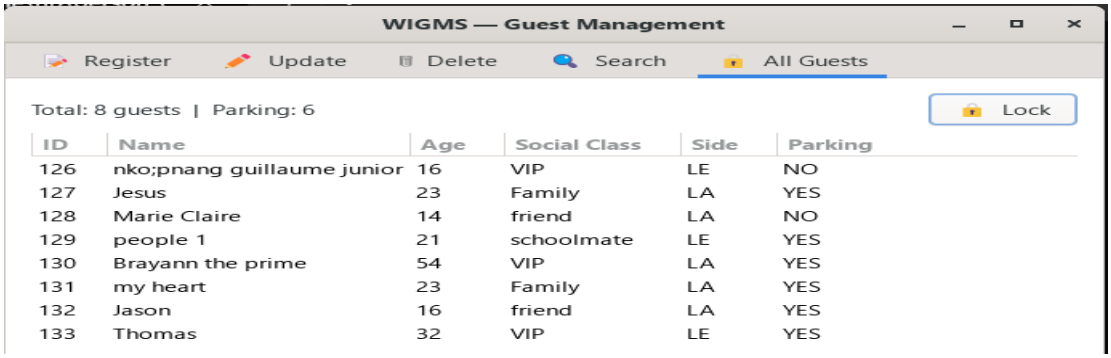
- **Search:** Here is an option where you can look for somebody in the list if guest by looking for his name or his social class in the search bar. It can be use to group the people by social class or by name.



- **All guests:** this class give to a user the access to all the guests which were registered in the program, the number of Guests which were stocked in the program, the ID of all the guests that where created in the application and the total number of parking spot assigned to guests. Since the ID and the information about the different Guests are confidential, we thought that it will be wise to protect them with a password which is "pokemon" in order to assure the protection of their data. The presentation of this page is like this:



All guests



ID	Name	Age	Social Class	Side	Parking
126	nko;pnang guillaume junior	16	VIP	LE	NO
127	Jesus	23	Family	LA	YES
128	Marie Claire	14	friend	LA	NO
129	people 1	21	schoolmate	LE	YES
130	Brayann the prime	54	VIP	LA	YES
131	my heart	23	Family	LA	YES
132	Jason	16	friend	LA	YES
133	Thomas	32	VIP	LE	YES

This interface displays all registered guests, including their IDs and assigned parking spaces. Access is protected by a password for security.

1.3 Person Management(Output)

The displayed results confirm that the data is correctly stored in and retrieved from the database. It also shows that any changes made by the user are immediately reflected, indicating proper interaction between the application logic and the database. Overall, the output validates the reliability and correctness of the Person module.

```
======= WIGMS SYSTEM =======
1. Register a Person
2. Display Last Registered
0. Exit
Choice: 1

- - - Guest Input - - -
ENter an ID
01

- - - Guest Name - - -
Enter a name
FRANCK

- - - Guest AGE - - -
ENter your Age
23

- - - Guest Social Class - - -
Enter your Social Class
VIP

 - - -Guest side- - -
Enter 0 for LE
Enter 1 for LA
0

 - - -PARKING- - -

 - - - PARKING - - -
Standard Access: You have been assigned ID: 212

======= WIGMS SYSTEM =======
1. Register a Person
2. Display Last Registered
0. Exit
Choice: 2

- - - Guest details- - -
ID:1     | Age:23        |side 'LE'     | Name:FRANCK
 | Social_class:VIP

======= WIGMS SYSTEM =======
```

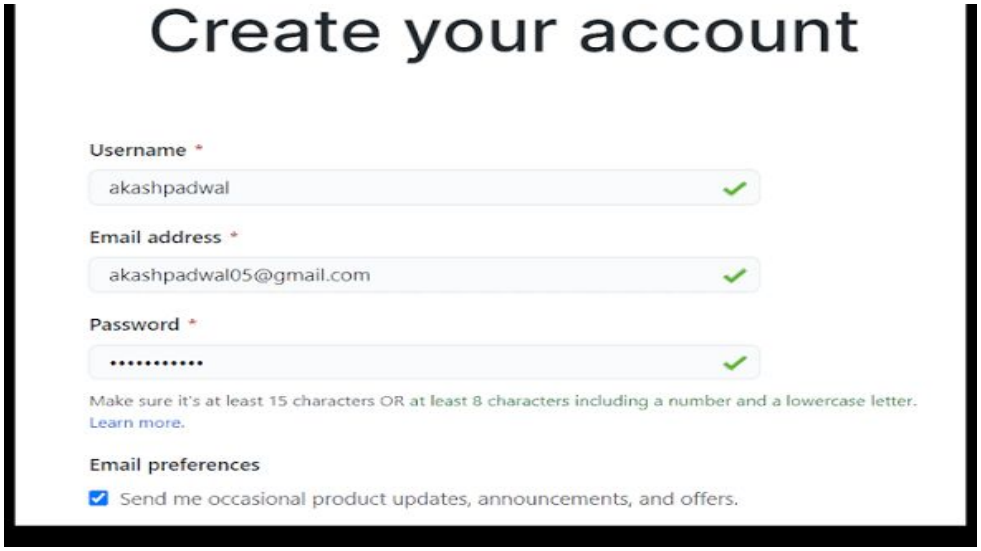
Figure 13: This output represents the successful execution of the Person module. It demonstrates that the system can effectively manage individual records by performing all core operations such as adding, viewing, updating, and deleting persons.

1.4 Setting Up a GitHub Repository

To create a GitHub repository using the web interface, follow these steps:

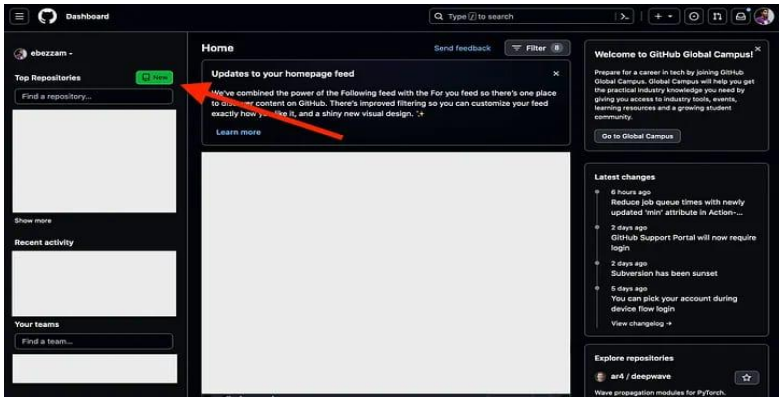
Step 1: Creating the GitHub accounts

To sign up for an account, navigate to https://github.com/ and follow the prompts. To keep your GitHub account secure you should use a strong and unique password.



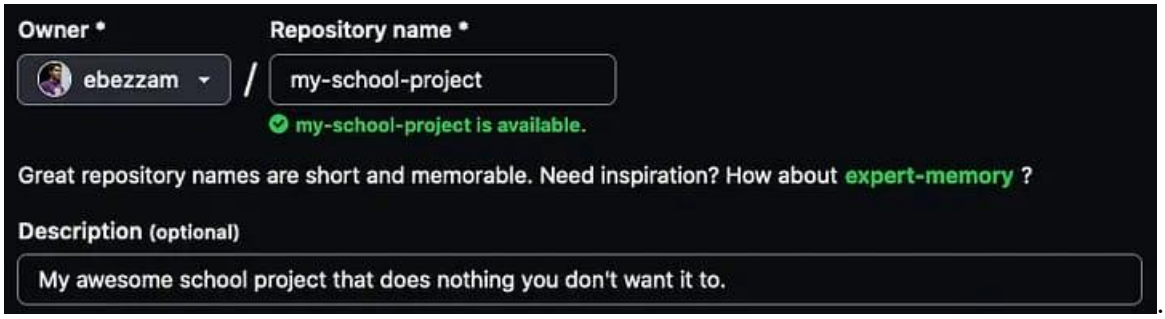
Step 2: Creating the Repository

Start a new repository in the upper-right corner of any page, select the plus (+) icon, then click New repository



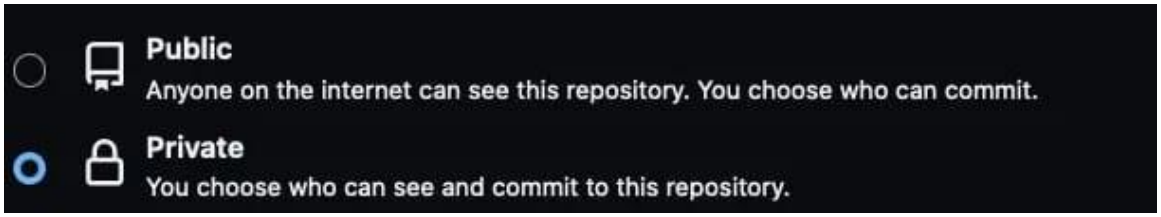
Step 3: Specify an owner

If you belong to any organizations, use the "Owner" dropdown menu to select the account (your personal account or an organization) that will own the repository. Enter a name — type a short, memorable name for your repository in the "Repository name" box.



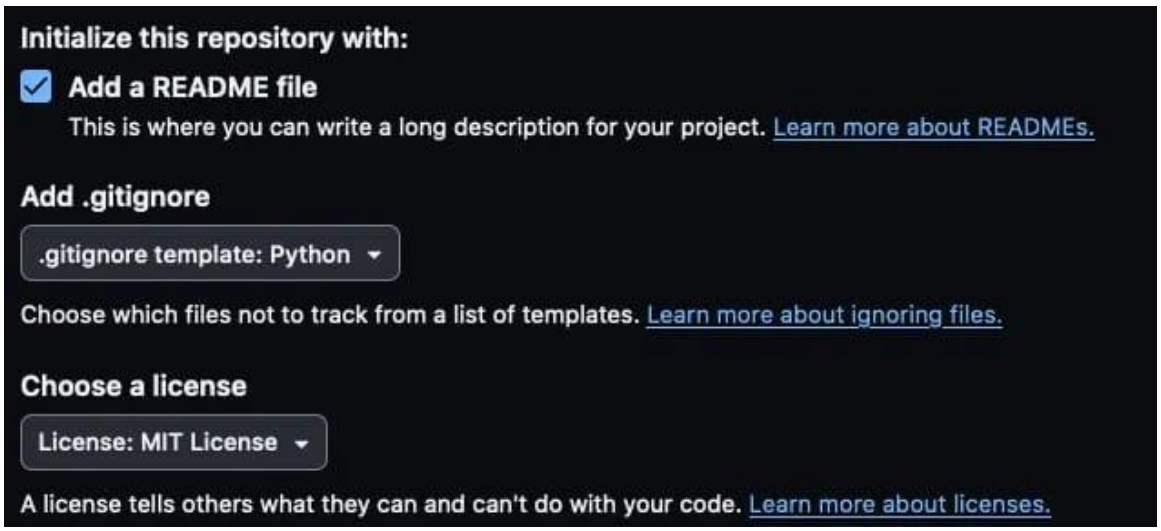
Step 4: Choose repository visibility

Decide whether the repository will be Public (anyone can see it) or Private (only you and collaborators you choose can access it).



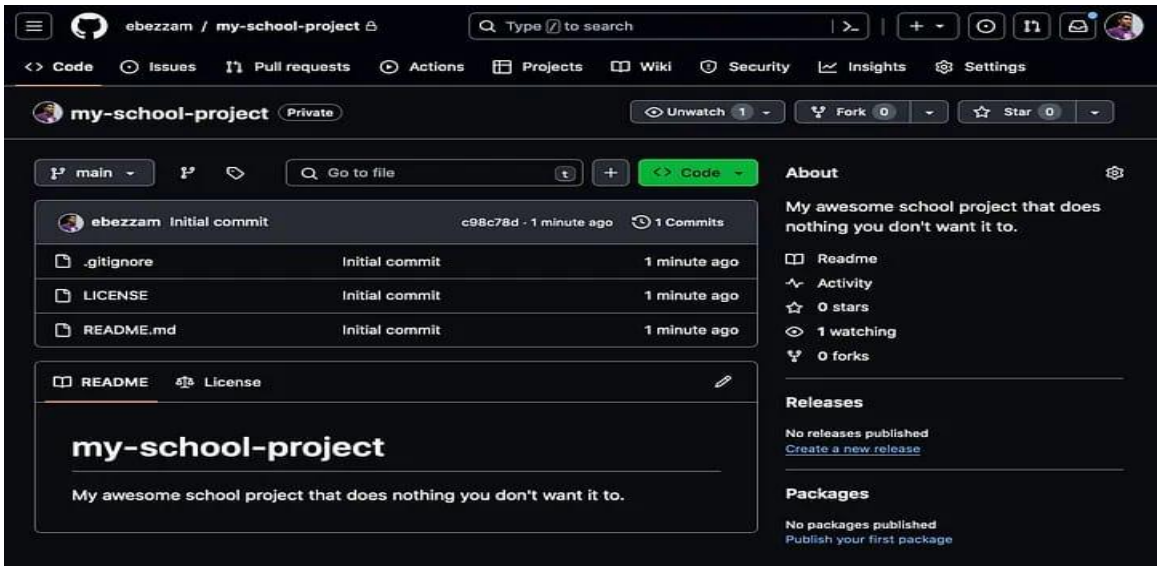
Step 5: Initialize the repository

Select Add a README file to create a document that describes your project. You can also optionally add a .gitignore file (to exclude specific file types from version control) and a license.



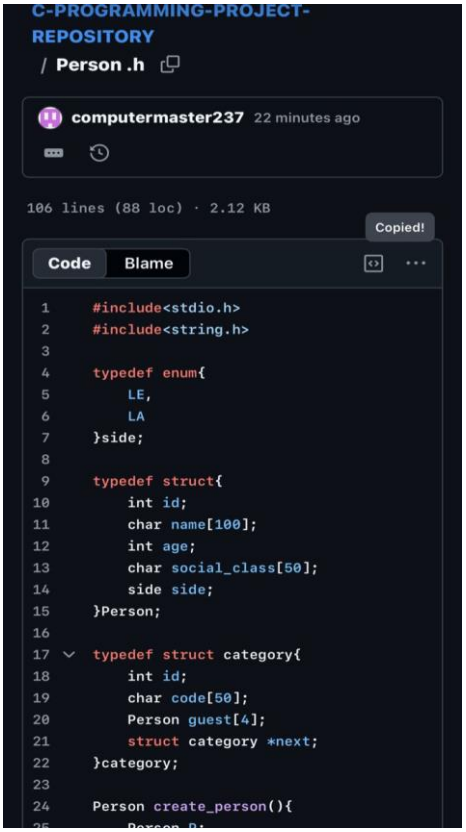
Step 6: Create the repository

Click the green Create repository button at the bottom of the page.



Step 7: Start using your repository

GitHub repository is now created and ready for you to start adding files and managing your project with version control. You can then clone the repository to your local machine using the provided URL to work with it locally.

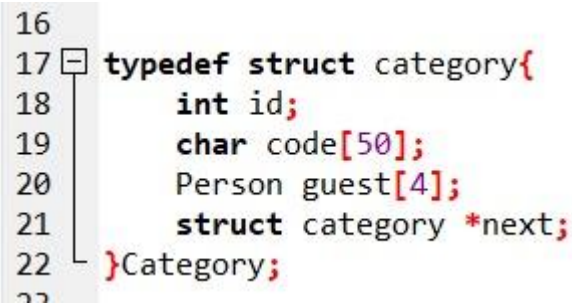


MODULE 2: CATEGORY MANAGEMENT

2.1 Objectives

- Organize guests into categories
- Use linked lists for dynamic storage
- Improve system structure

2.2 Category Structure



```
typedef struct category{
    int id;
    char code[50];
    Person guest[4];
    struct category *next;
}Category;
```

Figure 14: This image shows the structure used to represent a category. It stores information about different types of guests such as VIP, family, or friends.

2.3 Implemented Functions

2.3.1 Create_Category

```c
Category* create_category() {
    Category* new_node = (Category*)malloc(sizeof(Category));
    if (new_node == NULL) return NULL;

    printf("Enter Category ID: ");
    scanf("%d", &new_node->id);
    printf("Enter Category Code: ");
    scanf("%s", new_node->code);

    new_node->next = NULL;
    return new_node;
}

void insert_category(Category **tete) {
    Category* new_cat = create_category();
    if (new_cat) {
        new_cat->next = *tete;
        *tete = new_cat;
    }
}
```

Figure 15: This code represents the function used to create and store a new category.

2.3.2 Update_category

```c
void update_category(Category *tete, int id) {
    Category *current = tete;
    while (current != NULL) {
        if (current->id == id) {
            printf("Enter New Code: ");
            scanf("%s", current->code);
            return;
        }
        current = current->next;
    }
}
```

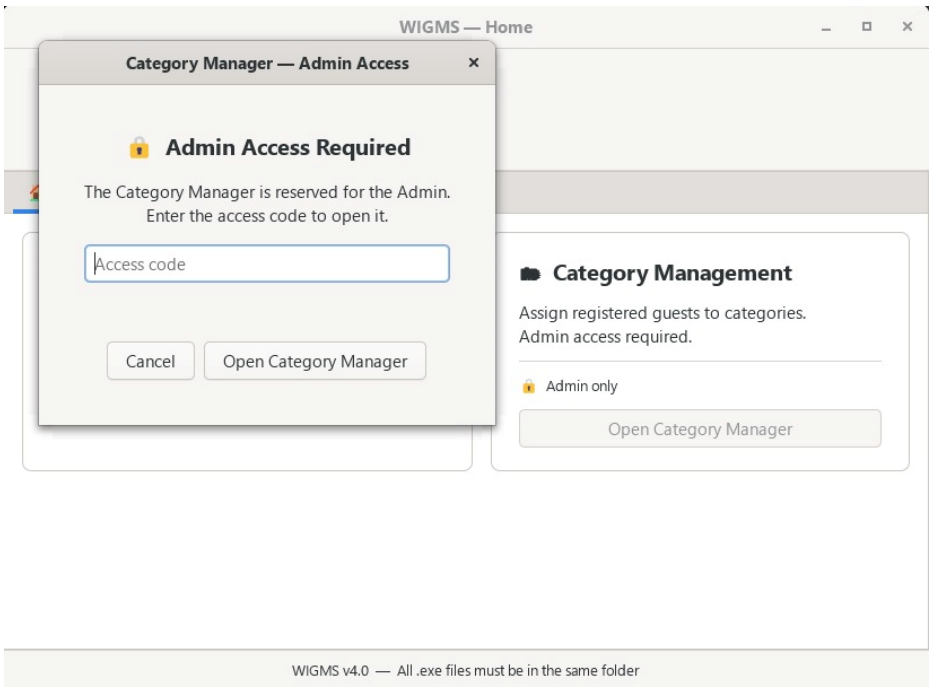
Figure 16: This function is used to create a new category and store it in the system.

2.3.3 Delete_category

```c
void delete_category(Category **tete, int id) {
    Category *temp = *tete, *prev = NULL;
    if (temp != NULL && temp->id == id) {
        *tete = temp->next;
        free(temp);
        return;
    }
    while (temp != NULL && temp->id != id) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) return;
    prev->next = temp->next;
    free(temp);
}
```

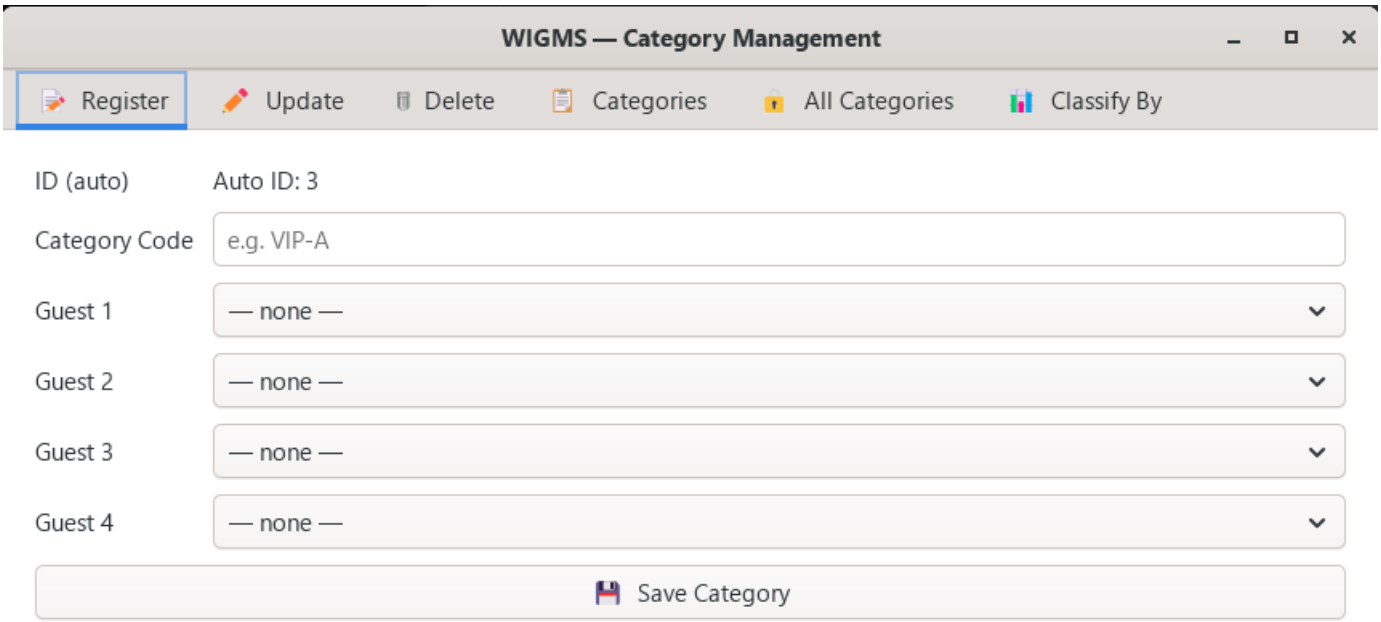
Figure 17: This code deletes a category from the system and updates the linked list accordingly.

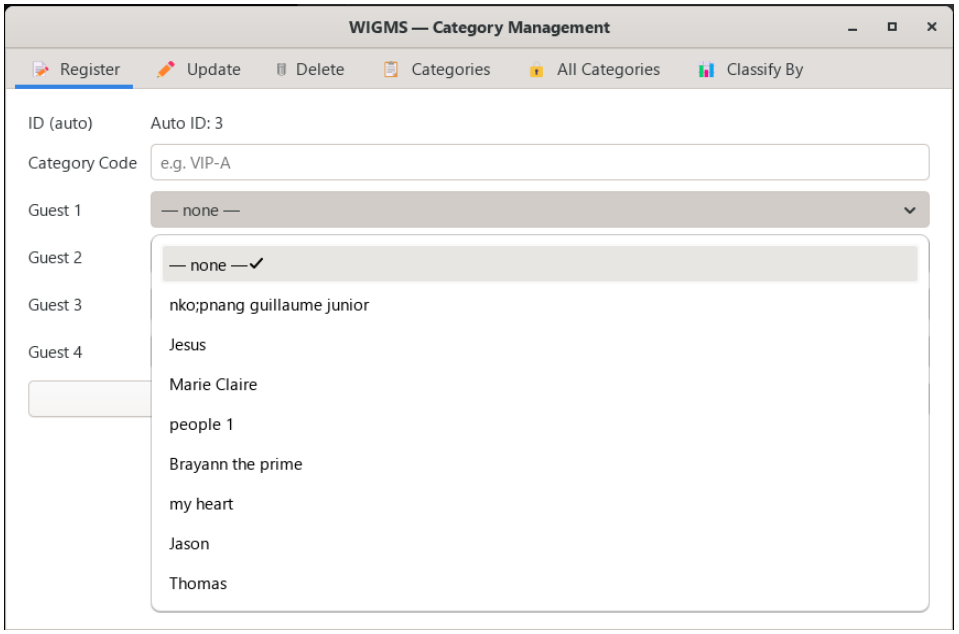
2.4 Category's Graphical User Interface



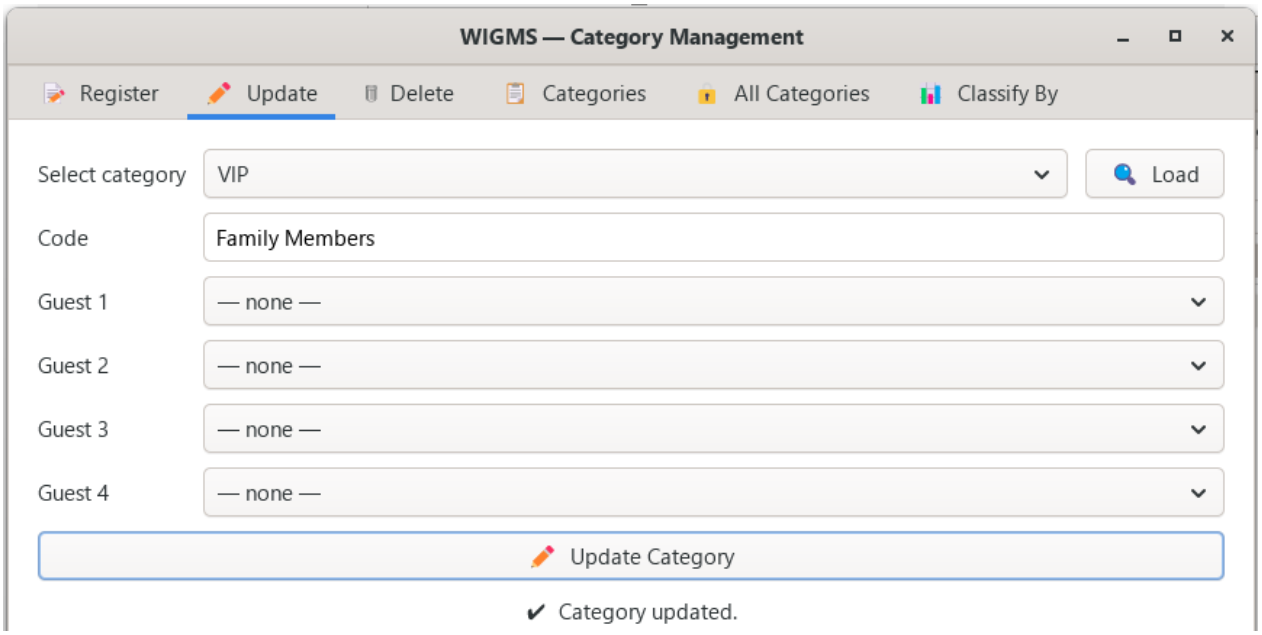
In the category module, we are handling the different kind of people which could come to register themselves: the friend, the family, the VIP guests, the partners, etc. Here, there are a lot of tasks which can be done, all subdivided into Classes. They are:

- **Register:** here, we are registering the different category of people which will be used for the organization of the ceremony. Here, we linked the person and the category module in order to enable the different guest which are already register to be automatically assign to an existent category. We also made ourselves sure to let the ID of each category to be automatically generated in order to leave redundancies and promote privacy in the program

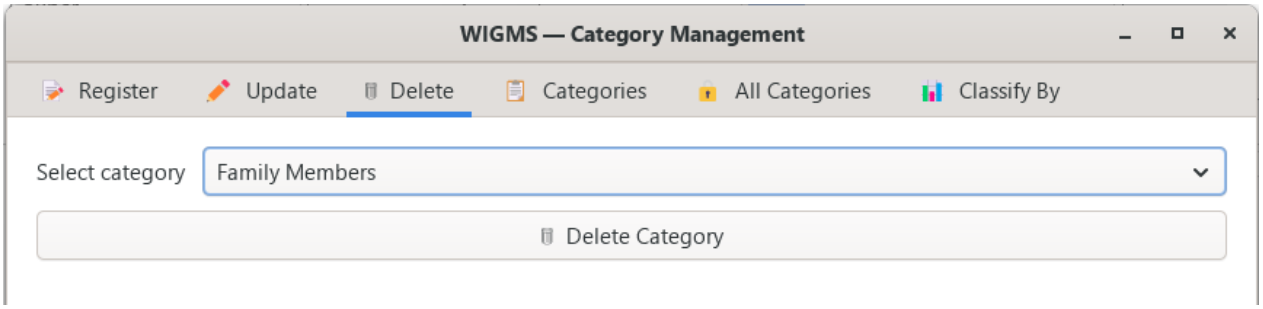




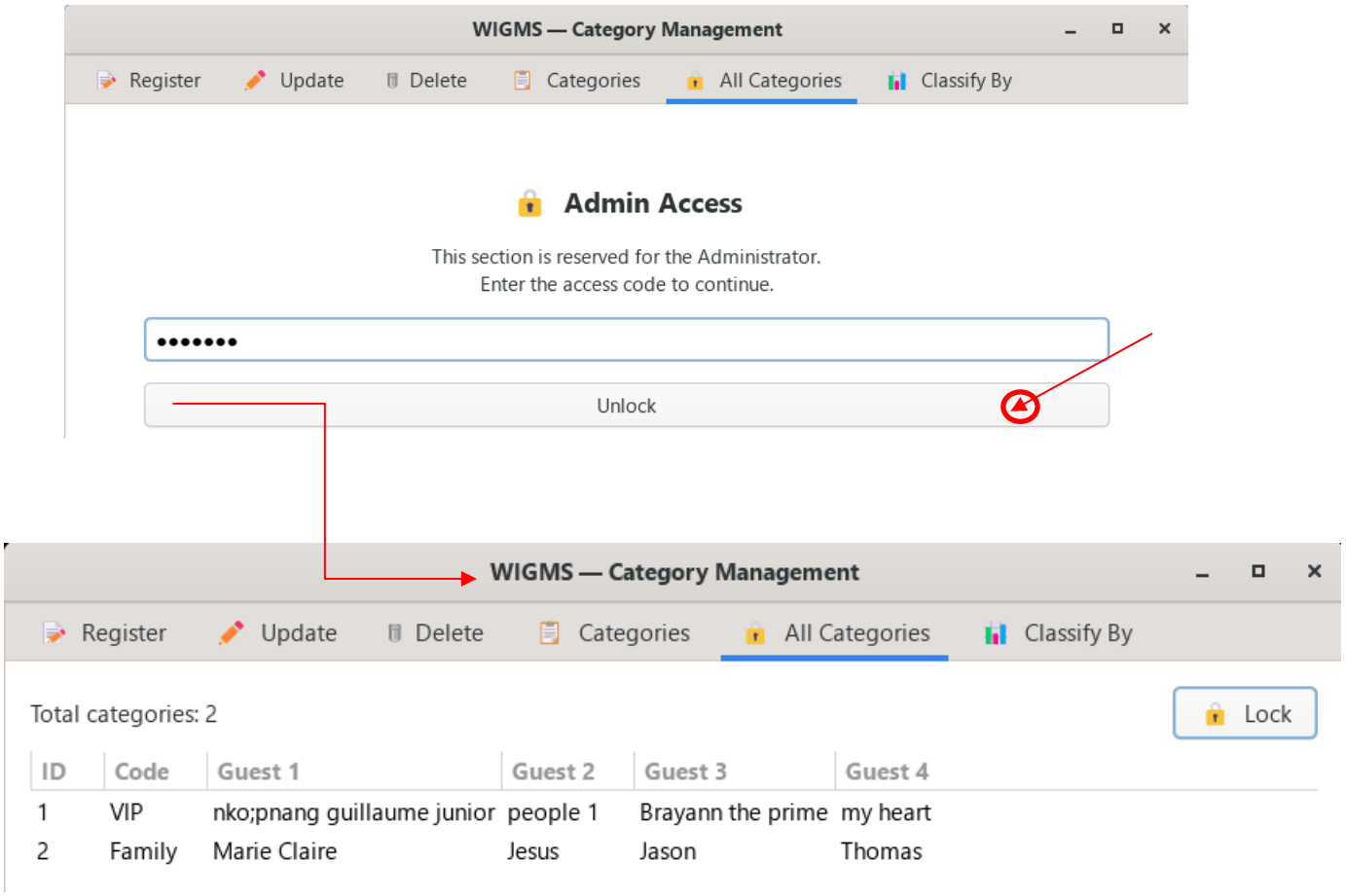
- **Update:** here we have the ability to modify the information on each category just by entering the name of the of the category in the research space and just modifying the information which are already inside the category description. Like here, we change the already existant VIP category into family members



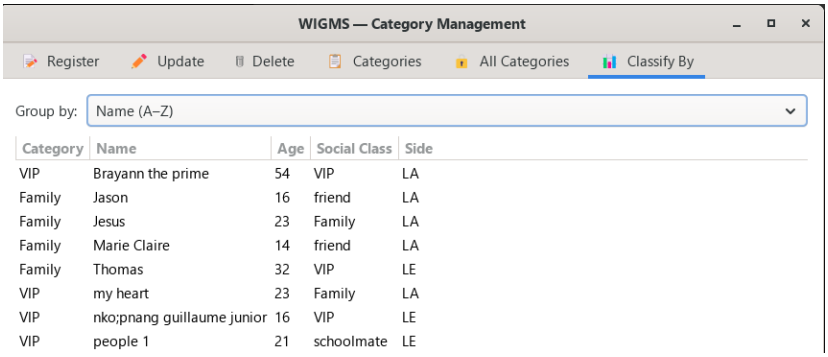
- **Delete:** It's an option where it's possible for an user to delete an existant category and the contant of these in the case that you have created an category which where not supposed to exist. It's like on this picture, where you just have the charge of write the name of the category and select delete.

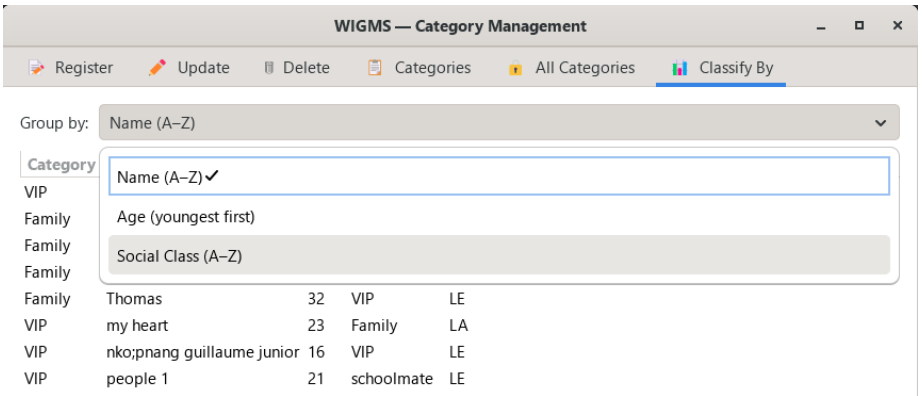


- **All categories:** this class give to a user the access to all the different categories which were generated by the program, the total number of categories, and the number of Guests which were assigned to a category, the ID of all the categories that where created in the application. Since the ID and the information about the different Guests and categories are confidential, we thought that it will be wise to protect them with a password which is "pokemon" in order to assure the protection of their data. The presentation of this page is like this:



- **Classify by:** here, we are a bit treating the functionalities of the third module. In this class, we are classify the entered by Guest according to some criteria like the name, the age or the social class in order to prioritize some category during the ceremony towards other one. It's as we can below:





2.5 Linked List Explanation

Categories are stored using a linked list:

- Each node represents a category
- Nodes are connected using pointers
- Allows dynamic insertion and deletion

2.6 Category Management(Output)

```
--- WIGMS: Category Management Test ---

1. Insert Category
2. Display All
3. Update Category
4. Delete Category
5. Total Guest Count
6. Exit
Choice: 1
Enter Category ID: 01
Enter Category Code: A01

1. Insert Category
2. Display All
3. Update Category
4. Delete Category
5. Total Guest Count
6. Exit
Choice: 1
Enter Category ID: 02
Enter Category Code: A02

1. Insert Category
2. Display All
3. Update Category
4. Delete Category
5. Total Guest Count
6. Exit
Choice: 3
Enter ID to update: 01
Enter New Code: 03
```

Figure 18: This output shows the proper functioning of the Category module within the system. It confirms that categories can be created, displayed, modified, and removed successfully.

MODULE 3: PRIORITY MANAGEMENT

3.1 Objectives of The Module

The main objective of this module is to:

- Introduce a priority system for managing guests or categories
- Implement a sorting algorithm (Merge Sort)
- Organize data based on importance
- Improve system efficiency and usability

3.2 Description of the Module

Priority Management allows the system to classify and organize elements according to their level of importance.

Each guest or category is assigned a priority level such as:

- Priority 1 → High (VIP guests)
- Priority 2 → Medium (Family members)
- Priority 3 → Low (Friends and others)

This classification helps ensure that important guests or items are handled first.

3.3 Implemented Functions

3.3.1 Prioritize by Age and Social class

```
void prioritize_by_age(Category **tete) {
    *tete = merge_sort_categories(*tete, SORT_BY_AGE);
}

void prioritize_by_social_class(Category **tete) {
    *tete = merge_sort_categories(*tete, SORT_BY_CLASS);
}
```

Figure 19: This code implements the prioritization logic based on age and social class. It assigns priority levels to guests, ensuring that more important individuals such as VIPs or older guests are handled first in the system.

3.3.2 Sorting by Age and Social class

```c
Category* sorted_merge(Category *a, Category *b, int criteria) {
    Category *result = NULL;
    if (a == NULL) return b;
    else if (b == NULL) return a;

    int condition = 0;
    if (criteria == SORT_BY_AGE) {

        condition = (a->guest[0].age <= b->guest[0].age);
    } else {

        condition = (strcmp(a->guest[0].social_class, b->guest[0].social_class) <= 0);
    }

    if (condition) {
        result = a;
        result->next = sorted_merge(a->next, b, criteria);
    } else {
        result = b;
        result->next = sorted_merge(a, b->next, criteria);
    }
    return result;
}

Category* merge_sort_categories(Category *head, int criteria) {
    if (head == NULL || head->next == NULL) return head;

    Category *a;
    Category *b;

    front_back_split(head, &a, &b);

    a = merge_sort_categories(a, criteria);
    b = merge_sort_categories(b, criteria);

    return sorted_merge(a, b, criteria);
}
```

Figure 20: This code represents the sorting mechanism used to organize guests according to selected criteria such as age or social class. It ensures that data is arranged in a meaningful order for better decision-making.

3.3.3 Main function

```c
int main() {

    Category *wedding_list = NULL;
    int choice;

    printf("--- WIGMS: Priority Management (DPR) Test ---\n\n");


    printf("Please add at least 3 categories to test sorting.\n");
    insert_category(&wedding_list);
    insert_category(&wedding_list);
    insert_category(&wedding_list);

    while (1) {
        printf("\n1. Display List\n2. Sort by Age (DPR)\n3. Sort by Social Class (DPR)\n4. Exit\n");
        printf("Selection: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                display_all_guests(wedding_list);
                break;
            case 2:
                prioritize_by_age(&wedding_list);
                printf("List sorted by age.\n");
                break;
            case 3:
                prioritize_by_social_class(&wedding_list);
                printf("List sorted by social class.\n");
                break;
            case 4:
                return 0;
        }
    }
```

Figure 21: This figure shows the main function of the program, which serves as the entry point of the application. It controls the execution flow by calling different functions related to prioritization and sorting.

3.4 Priority's Graphic User Interface

3.4.1 Merge Sort Categories

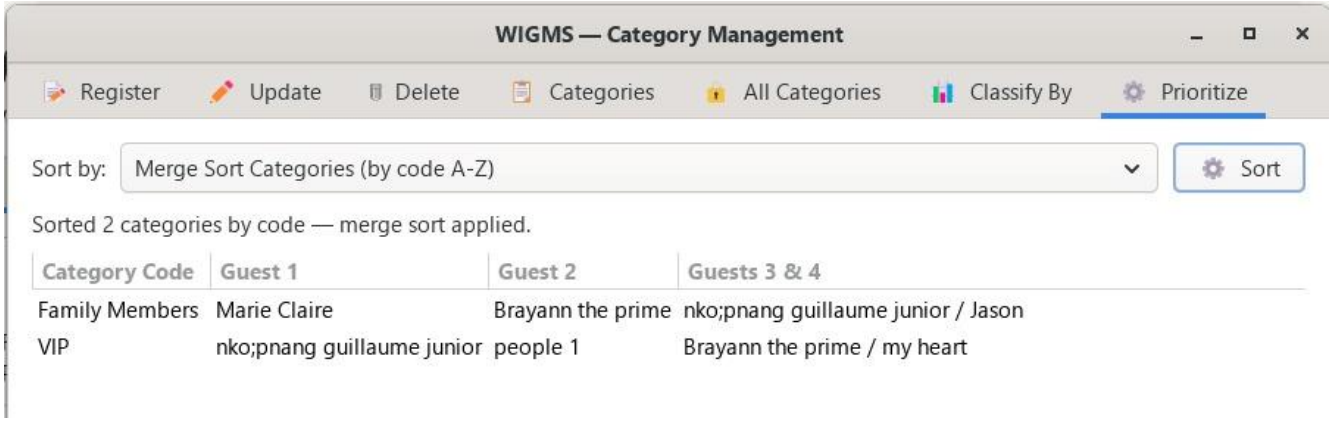


Figure 22: This interface allows users to apply the Merge Sort algorithm on categories. It visually demonstrates how categories are arranged based on priority or selected criteria.

3.4.2 Prioritize by Age

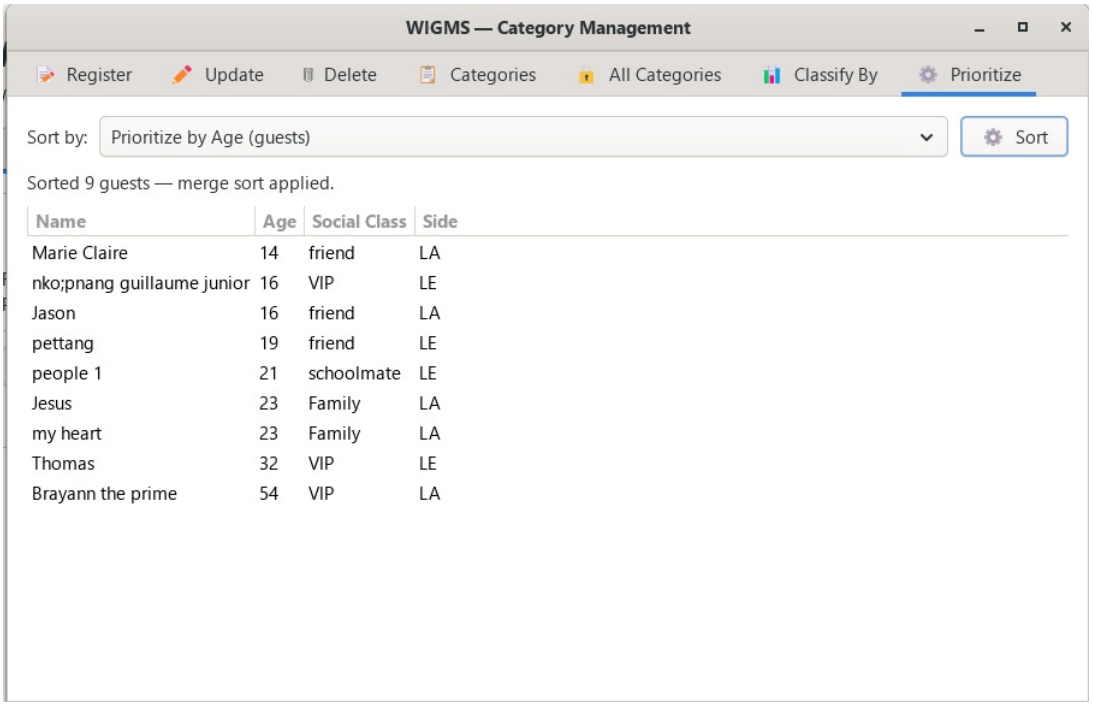


Figure 23: This interface enables users to prioritize guests based on their age. Older or more significant guests are displayed first depending on the sorting logic applied.

3.4.3 Prioritize by Social Class

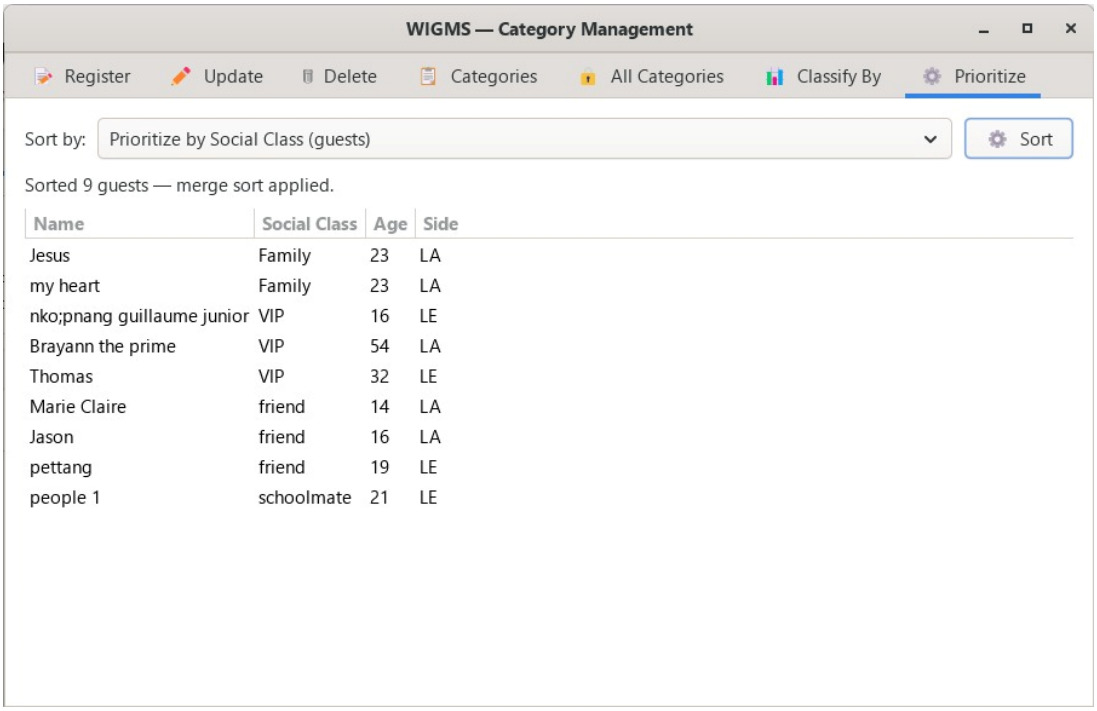


Figure 24: This interface allows prioritization based on social class (e.g., VIP, family, friends). It helps the system organize guests according to their importance level.

3.5 Priority Management (Console output)

The menu provides different operations such as displaying the list of categories, sorting by age, and sorting by social class using the Dynamic Priority Ranking (DPR) approach. After selecting the display option, the system outputs the list of categories along with their associated guest slots.

The result illustrates how categories are organized and displayed, confirming that the system correctly stores and retrieves data before applying any sorting algorithm.

```
--- WIGMS: Priority Management (DPR) Test ---

Please add at least 3 categories to test sorting.
Enter Category ID: 01
Enter Category Code: A01
Enter Category ID: 02
Enter Category Code: A02
Enter Category ID: 03
Enter Category Code: A03

1. Display List
2. Sort by Age (DPR)
3. Sort by Social Class (DPR)
4. Exit
Selection: 1
Category [A03] Guests:
 - Slot 1
 - Slot 2
 - Slot 3
 - Slot 4
Category [A02] Guests:
 - Slot 1
 - Slot 2
 - Slot 3
 - Slot 4
Category [A01] Guests:
 - Slot 1
 - Slot 2
 - Slot 3
 - Slot 4

1. Display List
2. Sort by Age (DPR)
```

Figure 25: This figure shows the execution of the Priority Management module in a console-based interface. It demonstrates how the user inputs multiple categories with their respective IDs and codes in order to test the sorting functionality.

8. Repository

The GitHub repository was created and validated by all group members. The project can be accessed using the following link: https://github.com/C-programming-project-1/C-PROGRAMMING-PROJECT-REPOSITORY

9. Tools and Technologies Used

- . C Programming Language for software development

- GitHub platform for version control and team collaboration.

- GTK for Graphic User Interface Design

- UML for Use case, Sequence and Class diagram

- Standard C libraries such as stdio.h ,string.h , stdib.h and gtk\gtk.h

10. Difficulties Encountered and Proposed Solutions

During the development of the Wedding Invitation and Gift Management System, the group encountered several general challenges that affected both Module 1,Module 2 and Module 3. These challenges were related to programming concepts, system design, and teamwork. However, appropriate solutions were applied to ensure the successful progress of the project.

10.1 Difficulty in Managing Dynamic Data Structures

The implementation of linked lists in Module 2 introduced another challenge. Unlike static data structures such as arrays, linked lists require proper handling of memory allocation and pointer connections between nodes. Errors such as incorrect linking of nodes, loss of data, and program crashes occurred during testing.

Proposed Solution:

The group used diagrams to visualize how linked lists work and how nodes are connected. Step-by-step tracing of the code was performed to identify errors. Additional practice on linked lists helped improve understanding and correct implementation.

10.2 Difficulty in Handling User Input

Managing user input using functions like scanf () was problematic, especially when handling both strings and integers. Issues such as skipped inputs and incorrect data reading were encountered.

Proposed Solution:

The group reviewed proper input handling techniques and improved the structure of input statements. Additional validation checks were introduced to ensure correct data entry. Extensive testing was conducted to eliminate input errors.

10.3 Understanding Merge Sort

Some group members found it difficult to understand how the algorithm divides and merges data.

Proposed Solution:

The team studied step-by-step examples and used diagrams to visualize the process.

10.4 Integrating Sorting with Existing Modules

Combining Module 3 with Modules 1 and 2 created compatibility challenges.

Proposed Solution:

Each module was tested independently before integration, and adjustments were made to ensure smooth interaction.

11. Next Steps of the Project

The next phase of the project will focus on Module 4: Gift Management. This module will allow the system to record, display, and manage gifts received during the wedding.

It will include creating a gift structure, implementing functions to add, update, delete, and display gifts, and linking each gift to a specific guest. The module will also be tested to ensure proper functionality and integration with the existing system.

12.Conclusion

This fourth report highlights both the correction of Module 1 and Module 2 , and the implementation of Module 3 which has successfully introduced priority management into the system, enhancing its functionality and efficiency. The use of the Merge Sort algorithm demonstrates a deeper understanding of advanced programming concepts such as recursion and data organization.

Additionally, the introduction of a user-friendly interface has made the system more interactive and efficient.

This module lays a strong foundation for the continuation of the project;